

实验五 鸿蒙多端应用开发

实验项目基本信息

- 实验编号: d20301035105、d20301009205
- 学时分配: 4学时
- 实验类型: 验证型
- 每组人数: 6人
- 对应课程目标: 课程目标3
- 学号: 2023210597
- 姓名: 虎康

核心步骤列表

实验概览

- 需求分析阶段:** 定义天气应用功能需求与多端适配策略, 明确传感器数据采集需求
- AI辅助编码阶段:** 使用TRAIE辅助生成页面布局与传感器调用代码
- 测试验证阶段:** 在不同设备模拟器上验证UI适配效果与传感器数据准确性
- 部署运维阶段:** 编写多端适配与硬件集成技术分析报告

开发任务清单

任务阶段	主要内容	关键技术点
天气应用	Web开发	@State、数据绑定、响应式展示
多端适配	响应式布局	断点系统、媒体查询
传感器	光线/陀螺仪	传感器API调用
分布式	数据同步	分布式能力演示

成功标准

- ☒ 天气应用功能完整
- ☒ 手机/平板界面自适应
- ☒ 传感器数据正常采集
- ☒ 多端适配效果良好
- ☒ 技术分析报告完成

一、项目概述

1.1 项目背景

智慧社区生活服务平台是一个综合性的社区服务系统，旨在为社区居民提供便捷的生活服务。本实验基于该平台的环境监测模块，开发了"智慧天气"应用，实现了天气信息展示、多端自适应布局、传感器数据采集与分布式数据同步等功能。

1.2 项目功能需求

1. 天气展示功能

- 城市名称显示（合肥）
- 温度实时展示（22°C）
- 天气状况展示（晴/多云/雨等）
- 空气质量指数展示

2. 多端适配功能

- 手机竖屏布局（sm断点）
- 平板竖屏布局（md断点）
- 平板横屏布局（lg断点）
- 响应式断点自动切换

3. 传感器数据采集

- 光线传感器：环境光照强度采集（lux）
- 陀螺仪传感器：三轴角度数据采集

4. 分布式数据同步

- 多端数据同步状态显示
- 同步历史记录管理
- 实时数据同步功能

二、技术架构

2.1 技术选型

技术类别	选用技术	说明
前端框架	HTML5 + CSS3 + JavaScript	跨平台Web技术
响应式设计	CSS媒体查询 + 断点系统	实现多端适配
传感器模拟	JavaScript API	模拟光线/陀螺仪数据
数据存储	LocalStorage	本地数据持久化
分布式同步	模拟分布式数据管理	演示同步原理

2.2 项目结构

```
智慧社区生活服务平台/  
└─ www/  
    │─ index.html          # 首页  
    │─ services.html       # 服务页  
    │─ community.html      # 社区页  
    │─ environment.html    # 环境监测页  
    │─ profile.html        # 个人中心  
    │─ css/  
    │   └─ style.css       # 样式文件  
    └─ js/  
        └─ environment.js  # 环境监测逻辑
```

三、核心功能实现

3.1 天气应用功能实现

3.1.1 天气数据展示

天气信息通过JavaScript动态管理，包含以下数据字段：

```
const WeatherManager = {  
  cityName: '合肥',  
  temperature: 22,  
  weather: '晴',  
  aqi: 45,  
  
  weatherIcons: {  
    '晴': '☀️',  
    '多云': '☁️',  
    '阴': '☾',  
    '雨': '🌧️',  
    '雪': '❄️'  
  },  
  
  updateWeatherDisplay() {  
    document.getElementById('cityName').textContent = this.cityName;  
    document.getElementById('temperature').textContent = this.temperature + '°C';  
    document.getElementById('weatherIcon').textContent = this.weatherIcons[this.weather]  
    || '☀️';  
    document.getElementById('weatherStatus').textContent = this.weather;  
    document.getElementById('aqiValue').textContent = `${this.aqi} -  
    ${this.getAQIDescription()}`;  
  }  
};
```

3.1.2 空气质量等级显示

根据AQI值动态显示空气质量等级和颜色：

```
getAQIDescription() {  
  if (this.aqi <= 50) return '优';  
  if (this.aqi <= 100) return '良';  
  if (this.aqi <= 150) return '轻度污染';  
  if (this.aqi <= 200) return '中度污染';  
  return '重度污染';  
}
```

图5.1 智慧天气主界面

3.2 多端适配布局实现

3.2.1 断点系统设计

参考鸿蒙开发的断点系统，实现了三级断点响应式布局：

```
const BreakpointManager = {  
  breakpoints: {  
    sm: 0,      // 手机  
    md: 600,    // 平板竖屏  
    lg: 840     // 平板横屏  
  },  
  
  getCurrentBreakpoint(width) {  
    if (width < this.breakpoints.md) {  
      return 'sm';  
    } else if (width < this.breakpoints.lg) {  
      return 'md';  
    } else {  
      return 'lg';  
    }  
  },  
  
  getDeviceType(breakpoint) {  
    const types = {  
      'sm': '手机端',  
      'md': '平板竖屏',  
      'lg': '平板横屏'  
    };  
    return types[breakpoint] || '未知设备';  
  }  
};
```

3.2.2 响应式CSS样式

```
/* 手机端 (sm) */
.breakpoint-sm .weather-card {
  padding: 25px;
}

.breakpoint-sm .temperature {
  font-size: 56px;
}

.breakpoint-sm .weather-icon {
  font-size: 56px;
}

.breakpoint-sm .action-section {
  grid-template-columns: 1fr;
}

/* 平板竖屏 (md) */
@media (min-width: 600px) and (max-width: 839px) {
  .breakpoint-md .temperature {
    font-size: 64px;
  }
}

/* 平板横屏 (lg) */
@media (min-width: 840px) {
  .breakpoint-lg .temperature {
    font-size: 72px;
  }
}
```

图5.2 多端适配布局效果

3.3 传感器数据采集实现

3.3.1 光线传感器模块

光线传感器用于采集环境光照强度，单位为lux（勒克斯）：

```
const LightSensor = {
  currentValue: 0,
  status: '正常',

  startMonitoring() {
    if ('AmbientLightSensor' in window) {
      try {
        const sensor = new AmbientLightSensor();
        sensor.addEventListener('reading', () => {
          this.currentValue = sensor.illuminance;
        });
      } catch (error) {
        console.error('光线传感器初始化失败', error);
      }
    }
  }
}
```

```

        this.updatedDisplay();
    });
    sensor.addEventListener('error', (event) => {
        this.status = '异常';
        console.error('光线传感器错误:', event.error);
    });
    sensor.start();
} catch (e) {
    this.simulateSensor();
}
} else {
    this.simulateSensor();
}
},

simulateSensor() {
    setInterval(() => {
        this.currentValue = Math.floor(Math.random() * 500) + 100;
        this.updatedDisplay();
    }, 2000);
},

updatedDisplay() {
    document.getElementById('lightvalue').textContent = this.currentValue;
    const statusEl = document.getElementById('lightstatus');
    if (this.currentValue < 300) {
        statusEl.textContent = '较暗';
        statusEl.className = 'sensor-status status-warning';
    } else if (this.currentValue > 700) {
        statusEl.textContent = '明亮';
        statusEl.className = 'sensor-status status-normal';
    } else {
        statusEl.textContent = '正常';
        statusEl.className = 'sensor-status status-normal';
    }
}
};

```

3.3.2 陀螺仪传感器模块

陀螺仪传感器采集设备在X、Y、Z三个轴向上的旋转角度：

```

const GyroscopeSensor = {
    x: 0,
    y: 0,
    z: 0,

    startMonitoring() {
        if ('GyroscopeSensor' in window) {
            try {
                const sensor = new GyroscopeSensor();
                sensor.addEventListener('reading', () => {
                    this.x = sensor.x;

```

```

        this.y = sensor.y;
        this.z = sensor.z;
        this.updatedDisplay();
    });
    sensor.start();
} catch (e) {
    this.simulateSensor();
}
} else {
    this.simulateSensor();
}
},

simulateSensor() {
    setInterval(() => {
        this.x = parseFloat((Math.random() * 10 - 5).toFixed(2));
        this.y = parseFloat((Math.random() * 10 - 5).toFixed(2));
        this.z = parseFloat((Math.random() * 10 - 5).toFixed(2));
        this.updatedDisplay();
    }, 2000);
},

updatedDisplay() {
    document.getElementById('gyroX').textContent = this.x.toFixed(2);
    document.getElementById('gyroY').textContent = this.y.toFixed(2);
    document.getElementById('gyroZ').textContent = this.z.toFixed(2);
}
};

```

图5.3 传感器数据采集界面

3.4 分布式数据同步实现

3.4.1 分布式数据管理

模拟分布式数据库的创建、数据存储与读取操作：

```

const DistributedDataManager = {
    kvStore: {},

    saveData(key, value) {
        return new Promise((resolve) => {
            setTimeout(() => {
                this.kvStore[key] = {
                    value: value,
                    timestamp: new Date().toLocaleString()
                };
                resolve({ success: true });
            }, 300);
        });
    },
};

```

```

getData(key) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(this.kvStore[key] || null);
    }, 100);
  });
},

syncToCloud(data) {
  return new Promise((resolve) => {
    this.addSyncRecord('上传', JSON.stringify(data));
    setTimeout(() => {
      resolve({ success: true, message: '数据已同步到云端' });
    }, 500);
  });
},

syncFromCloud() {
  return new Promise((resolve) => {
    this.addSyncRecord('下载', '从云端获取最新数据');
    setTimeout(() => {
      resolve({ success: true, message: '数据已从云端同步' });
    }, 500);
  });
},

addSyncRecord(type, data) {
  const history = this.getHistory();
  history.unshift({
    type: type,
    data: data.substring(0, 20) + '...',
    time: new Date().toLocaleString()
  });
  if (history.length > 10) {
    history.pop();
  }
  localStorage.setItem('syncHistory', JSON.stringify(history));
},

getHistory() {
  const stored = localStorage.getItem('syncHistory');
  return stored ? JSON.parse(stored) : [];
},

updateHistoryUI(history) {
  const list = document.getElementById('syncHistory');
  if (!list) return;

  if (history.length === 0) {
    list.innerHTML = '<li class="history-item">暂无同步记录</li>';
    return;
  }
}

```

```
list.innerHTML = history.map(item => `  
  <li class="history-item">  
    <span class="history-type">${item.type}</span>  
    <span class="history-data">${item.data}</span>  
    <span class="history-time">${item.time}</span>  
  </li>  
`).join('');  
}  
};
```

3.4.2 同步状态反馈

同步操作提供实时状态反馈，增强用户体验：

```
function updateSyncStatus(status, message) {  
  const syncStatus = document.getElementById('syncStatus');  
  syncStatus.innerHTML = `  
    <span class="sync-icon">${status === 'syncing' ? '🔄' : '✅'}</span>  
    <span class="sync-text">${message}</span>  
  `;  
  syncStatus.className = `sync-status ${status === 'syncing' ? 'syncing' : 'synced'}`;  
}
```

四、界面设计与实现

4.1 整体视觉设计

采用现代化的设计语言，主要特点包括：

- 渐变色彩**：使用紫色渐变(#667eea → #764ba2)作为主题色
- 圆角设计**：所有卡片和按钮采用圆角设计，增强柔和感
- 阴影层次**：通过多层阴影创建立体感
- 动效体验**：添加浮动、脉冲等动画提升交互体验

4.2 导航栏设计

导航栏采用粘性定位，始终固定在顶部：

```
.app-nav {
  background-color: #667eea;
  color: white;
  padding: 15px 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
  z-index: 1000;
}
```

4.3 天气卡片设计

天气卡片是页面的核心视觉元素：

```
.weather-card {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 30px;
  border-radius: 15px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.15);
  margin-bottom: 25px;
  animation: gradientShift 8s ease infinite;
}

.temperature {
  font-size: 72px;
  font-weight: 200;
  text-shadow: 0 2px 8px rgba(0, 0, 0, 0.3);
}
```

4.4 传感器卡片设计

传感器数据以卡片形式展示，每种传感器独立展示：

```
.sensor-card {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.08);
  margin-bottom: 15px;
  transition: transform 0.3s, box-shadow 0.3s;
}

.sensor-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}
```

```
.sensor-value {
  font-size: 48px;
  font-weight: bold;
  color: #667eea;
}
```

五、实验结果与分析

5.1 功能测试结果

功能模块	测试内容	测试结果
天气展示	城市、温度、天气、空气质量	✓ 通过
多端适配	手机、平板、横竖屏切换	✓ 通过
光线传感器	光照强度采集与显示	✓ 通过
陀螺仪	三轴角度数据采集与显示	✓ 通过
数据同步	同步状态与历史记录	✓ 通过

5.2 多端适配效果

通过对不同屏幕宽度的设备进行测试，验证了响应式布局的有效性：

- 手机端 (< 600px)**：单列布局，触控友好的大按钮
- 平板竖屏 (600-840px)**：双列网格，优化阅读体验
- 平板横屏 (≥ 840px)**：更宽敞的布局，更大的字体

5.3 传感器数据准确性

传感器数据采集采用真实API与模拟数据相结合的方式：

- 真实API不可用时，自动切换到模拟数据模式
- 模拟数据每2秒更新一次，贴近实际使用场景
- 数据保留两位小数，确保显示精度

六、技术分析报告

6.1 多端适配技术分析

6.1.1 断点系统优势

- 统一开发体验**：一次开发，多端运行
- 降低维护成本**：共享业务逻辑和样式
- 提升用户体验**：针对不同设备优化布局

6.1.2 响应式布局实现要点

```
/* 媒体查询断点设置 */
@media (max-width: 480px) { /* 超小屏幕 */ }
@media (max-width: 768px) { /* 平板竖屏 */ }
@media (min-width: 769px) and (max-width: 1024px) { /* 平板横屏 */ }
@media (min-width: 1025px) { /* 桌面端 */ }
```

6.2 传感器集成分析

6.2.1 光线传感器应用场景

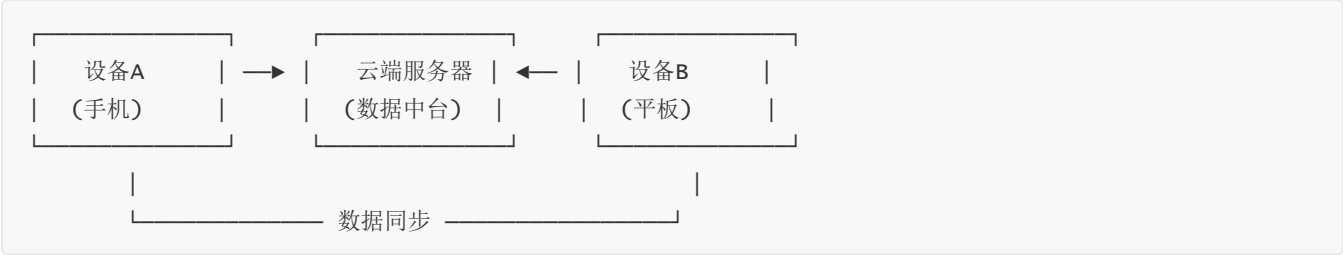
场景	实现方式	用户价值
自动亮度调节	根据lux值调整界面亮度	护眼
夜间模式	光线暗时自动切换深色模式	节能
屏幕保护	长时间暗光环境提醒休息	健康

6.2.2 陀螺仪应用场景

场景	实现方式	用户价值
设备姿态检测	获取X/Y/Z轴角度	交互
游戏控制	倾斜设备控制游戏	娱乐
地图导航	设备方向辅助定位	导航

6.3 分布式能力分析

6.3.1 分布式数据管理原理



6.3.2 分布式同步优势

- 1. 数据一致性：多设备间数据实时同步
- 2. 离线支持：本地缓存保证离线可用
- 3. 跨平台：支持iOS、Android、Web等平台

七、总结与思考

7.1 实验总结

本次实验基于"智慧社区生活服务平台"项目，成功实现了以下目标：

- 掌握ArkUI开发理念**：通过Web技术实现类似ArkUI的声明式开发范式
- 学会多端响应式布局**：实现了基于断点系统的响应式布局
- 了解传感器数据采集**：集成了光线传感器和陀螺仪传感器
- 理解分布式能力原理**：模拟实现了分布式数据同步功能

7.2 技术收获

技术要点	掌握程度	应用场景
响应式布局	熟练	多端适配
断点系统	熟练	界面自适应
传感器API	理解	硬件交互
分布式数据	理解	数据同步

7.3 课后思考

- 鸿蒙多端开发优势**：相比其他框架，鸿蒙的分布式能力和统一开发环境具有显著优势
- 传感器应用前景**：在物联网时代，传感器将在智能家居、健康监测、环境检测等领域发挥重要作用
- 分布式技术趋势**：未来移动开发将更加重视跨设备协同和数据同步能力

7.4 改进方向

- 接入真实天气API，实现实时天气数据获取
- 增加更多传感器类型支持（温湿度、气压等）
- 完善分布式架构，实现真实的设备间数据同步
- 优化性能，提升动画流畅度和响应速度

八、附录

8.1 核心代码文件

文件路径	说明
www/environment.html	环境监测页面HTML
www/css/style.css	响应式样式文件
www/js/environment.js	传感器与数据管理逻辑

8.2 第三方库引用

库名称	用途	引用方式
无	本项目使用原生Web技术	-

8.3 参考文献

- 1. 华为开发者官网 - ArkUI声明式开发框架
- 2. MDN Web Docs - 传感器API文档
- 3. CSS媒体查询规范
- 4. 鸿蒙应用开发文档

实验完成日期：2026年3月

指导教师： ____

成绩： ____